

[LG U+ Why not SW Camp 8기]

Team Moni

사용자 소비 패턴 분석
&이상 소비 탐지
핵심 로직 정의서

함수명 : mydata_start(request)

```
def mydata_start(request):
    moni_user = _current_moni_user(request)
    userid = moni_user.user_id
    name = moni_user.name

    has_spending = Spending.objects.filter(user=moni_user).exists()
    if not has_spending:
        df = data.generate_merged_user_data(userid)
        data.save_spending_df_to_db(df, moni_user)

        data.auto_categorize_spending_with_gemini(moni_user)
        data.sync_bank_accounts_from_report(moni_user)

        has_uncategorized =
Spending.objects.filter(user=moni_user).filter(
    Q(category__isnull=True) | Q(category='')).exists()

    if not has_uncategorized:
        feature = data.extract_feature_vector(moni_user)
        pred = data.use_classify_model(feature)
        type_obj = Type.objects.get(type_name=pred)

        moni_user.type = type_obj
        moni_user.save(update_fields=["type"])
        now = datetime.now()
        Notification.objects.create(user=moni_user,
notification_time=now, notification_detail=f"{name} 님, 소비 패턴이
달라졌어요")

    outlier.check_outlier(moni_user)
    return redirect("home")
```

- 사용자 최초 진입 시 소비 데이터 생성부터 소비 성향 분석, 이상 소비 탐지까지의 전체 분석 파이프라인을 실행하는 진입 함수

함수명 : check_notification(moni_user, today_date, detail)

```
def check_notification(moni_user, today_date, detail):
    with connection.cursor() as cursor:
        cursor.execute(
            """
            SELECT *
            FROM moni.notification
            WHERE user_id = %s
            AND DATE(notification_time) = %s
            AND notification_detail = %s
            LIMIT 1 """ ,
            [moni_user.user_id, today_date, detail])
        row = cursor.fetchone()
        return row
```

- 동일 날짜·동일 내용의 알림이 중복 생성되는 것을 방지하기 위한 검증 함수

함수명 : check_outlier(moni_user)

```
def check_outlier(moni_user):

    qs = Spending.objects.filter(user=moni_user).values()
    df = pd.DataFrame(list(qs))
    df = df[df["transaction_type"] == "출금"].copy()

    df["spend_date"] = (
        pd.to_datetime(df["spend_date"], utc=True)
        .dt.tz_convert("Asia/Seoul")
        .dt.tz_localize(None)
    )
    .
    .
    .

    past_daily = (
        df_past
        .groupby(["category", "day"])["price"]
        .sum()
        .reset_index()
    )

    box_stats = (
        past_daily
        .groupby("category")["price"]
```

```

        .agg(
            Q1=lambda x: x.quantile(0.25),
            Q3=lambda x: x.quantile(0.75),
        )
    )
    box_stats["IQR"] = box_stats["Q3"] - box_stats["Q1"]
    box_stats["upper_fence"] = box_stats["Q3"] + 0.5 * box_stats["IQR"]

    today_spending = (
        df[df["spend_date"].dt.date == target_date]
        .groupby("category")["price"]
        .sum()
        .reset_index(name="today_sum")
    )

    today_spending = today_spending.merge(
        box_stats[["upper_fence"]],
        on="category",
        how="left"
    )

    .
    .
    .

    if check_notification(moni_user, today_date, detail):
        return

    Notification.objects.create(
        user=moni_user,
        notification_detail=detail
    )

    return

```

- 과거 소비 분포를 기준으로 전일 소비 내역을 비교하여 통계적으로 비정상적인 소비를 탐지하고 사용자 알림을 생성

함수명 : generate_merged_user_data(user_id)

```
def generate_merged_user_data(user_id):
    config = PERSONAS[random.choice(list(PERSONAS.keys()))]
    transaction_list = []

    my_income = random.randint(*config['income_amount'])
    my_spending_bias = random.randint(-1, 1)
```

... 생성 로직 생략

```
return
pd.DataFrame(transaction_list).sort_values('trans_datetime').reset_index(drop=True)
```

사용자 페르소나 기반으로 수입, 고정 지출, 변동 지출, 이벤트성 지출을 통합한 현실적인 소비 데이터를 생성

함수명 : save_spending_df_to_db(df, user)

```
def save_spending_df_to_db(df, user):

    df["trans_datetime"] = pd.to_datetime(df["trans_datetime"])

    if df["trans_datetime"].dt.tz is None:
        df["trans_datetime"] =
df["trans_datetime"].dt.tz_localize('Asia/Seoul')
    else:
        df["trans_datetime"] =
df["trans_datetime"].dt.tz_convert('Asia/Seoul')

    records = []

    for _, row in df.iterrows():
        spend = Spending(
            user=user,
            spend_date=row['trans_datetime'],
            method=row['method'],
            price=row['amount'],
            details=row['merchant_name'],
            transaction_type=row['trans_type'],
            memo=""
        )
        records.append(spend)

    Spending.objects.bulk_create(records)
```

- Pandas 기반 소비 데이터를 ORM 객체로 변환하여 대량 저장하는 DB 적재 함수

함수명 : `extract_feature_vector(moni_user)`

```
def extract_feature_vector(moni_user):

    df = spending_to_dataframe(moni_user)
    # print("df : ", df.head(10))

    df["trans_dtime"] = pd.to_datetime(df["trans_dtime"])
    df["year_month"] = df["trans_dtime"].dt.strftime("%Y-%m")

    income_df = df[df["trans_type"] == "입금"]
    expense_df = df[df["trans_type"] == "출금"]

    income_month =
income_df.groupby("year_month")["amount"].sum().reset_index(name="income")
    expense_month =
expense_df.groupby("year_month")["amount"].sum().reset_index(name="expense")

    summary = income_month.merge(expense_month, how="outer",
on="year_month").fillna(0)
    summary["remain"] = summary["income"] - summary["expense"]

    saving_month = df[df["category"] ==
"savemoney"].groupby("year_month")["amount"].sum().reset_index(name="saving_amount")
    summary = summary.merge(saving_month, how="left",
on="year_month").fillna(0)

    invest_month = df[df["category"] ==
"investment"].groupby("year_month")["amount"].sum().reset_index(name="invest_amount")
    summary = summary.merge(invest_month, how="left",
on="year_month").fillna(0)

    fixed_month = df[df["category"].isin(["living",
"transport"])].groupby("year_month")["amount"].sum().reset_index(name="fixed_amount")
    month_df = summary.merge(fixed_month, how="left",
on="year_month").fillna(0)

    feature = {}
```

```

total_saving = month_df["saving_amount"].sum()
total_expense = month_df["expense"].sum()
feature["saving_rate"] = total_saving / total_expense if
total_expense > 0 else 0

avg_income = month_df["income"].mean()
avg_remain = month_df["remain"].mean()
feature["remain_ratio"] = avg_remain / avg_income if avg_income > 0
else 0

total_invest = month_df["invest_amount"].sum()
feature["invest_ratio"] = total_invest / total_expense if
total_expense > 0 else 0

exp_std = month_df["expense"].std()
exp_mean = month_df["expense"].mean()
feature["spend_volatility"] = exp_std / exp_mean if exp_mean > 0
else 0

feature["peak_spend_months"] = int((month_df["expense"] > exp_mean
* 1.5).sum())

total_fixed = month_df["fixed_amount"].sum()
feature["fixed_cost_ratio"] = total_fixed / total_expense if
total_expense > 0 else 0

return feature

```

- 월 단위 소비 패턴을 카테고리별 수치 특징으로 변환하여 모델 입력용 **Feature Vector**를 생성

함수명 : use_classify_model(feature)

```
def use_classify_model(feature):  
  
    feature_vector = pd.DataFrame([{k: float(feature[k]) for k in  
FEATURE_ORDER}])  
    loaded = joblib.load("model.joblib")  
    loaded_model = loaded["RandomForestmodel"]  
    encoder = loaded["Label_Encoder"]  
    scaler = loaded["RobustScaler"]  
  
    x_test_scaled = scaler.transform(feature_vector)  
    x_test = pd.DataFrame(x_test_scaled, columns=FEATURE_ORDER)  
    y_pred = loaded_model.predict(x_test)  
    y_pred = encoder.inverse_transform(y_pred)  
  
    return y_pred[0]
```

- 사전 학습된 소비 성향 분류 모델을 사용해 사용자 소비 유형을 예측